



Memory in the Age of AI Agents: A Survey

Forms, Functions and Dynamics

**Authors: Yuyang Hu, Shichun Liu, Yanwei Yue,
Guibin Zhang, et al. (多机构合作)**

**Affiliations: NUS, RUC, Fudan, PKU, NTU,
Tongji, UCSD, HKUST(GZ), etc.**

Memory is a core capability of LLM-based AI Agents, supporting long-horizon reasoning, continuous adaptation and environmental interaction.

Currently, agent memory research is fragmented: inconsistent definitions, diverse implementations and messy terminologies make traditional short/long-term memory taxonomies no longer applicable.

In this survey, we systematically review agent memory from three unified perspectives: **Forms**, **Functions**, **Dynamics**.

1. **Forms**: Classify memory into token-level, parametric and latent memory.
2. **Functions**: Divide into factual, experiential and working memory beyond simple time-based division.
3. **Dynamics**: Analyze the full lifecycle of memory formation, evolution and retrieval.

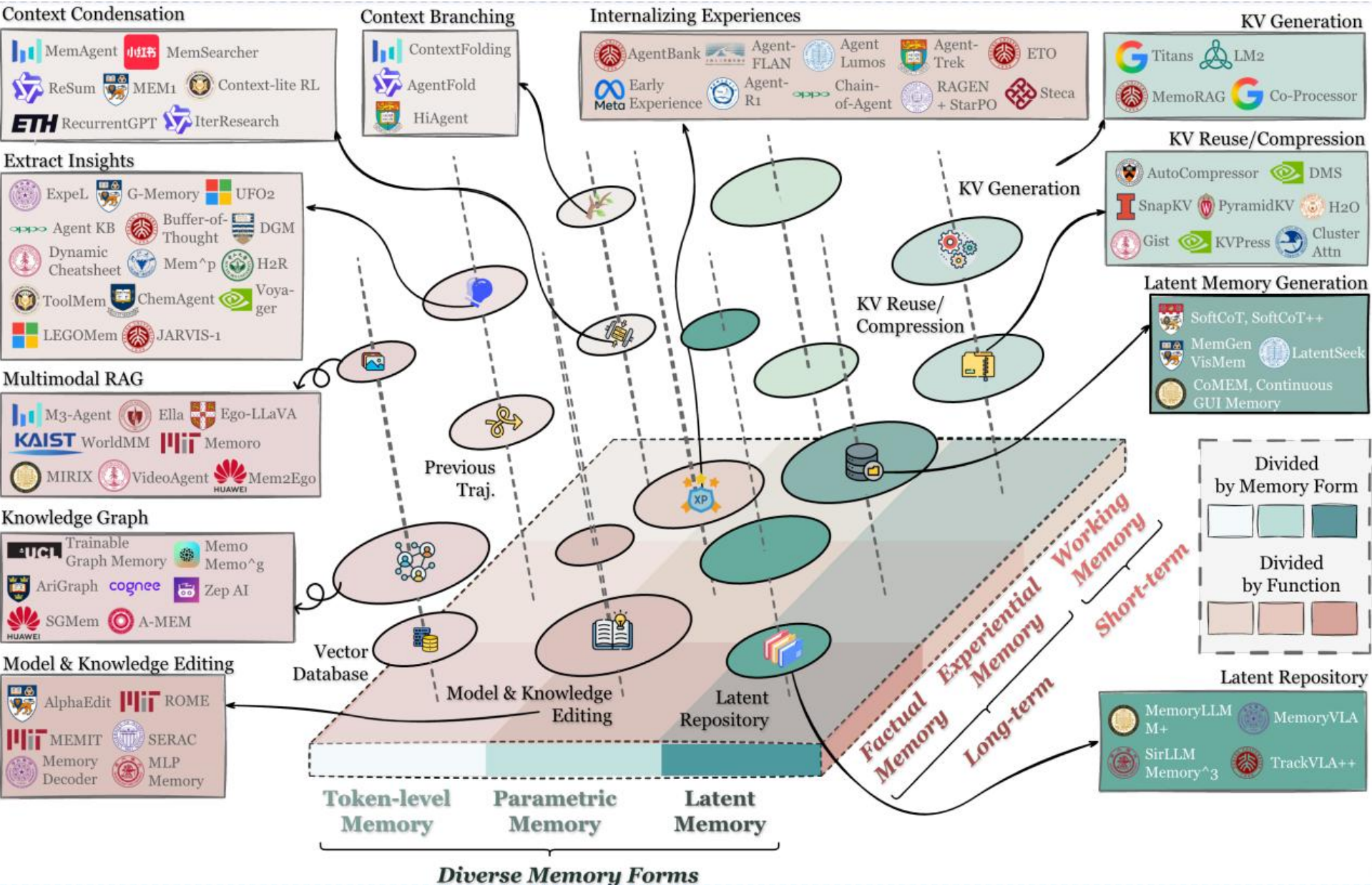
We also summarize existing benchmarks and open-source frameworks. Finally, we discuss key research frontiers including automated memory design, RL integration, multimodal/multi-agent memory and trustworthy memory.

This work aims to provide a unified framework and guidance for future agent memory research.

Introduction



北京大学
PEKING UNIVERSITY



Key Questions

- ❶ How is *agent memory* defined, and how does it relate to related concepts such as LLM memory, retrieval-augmented generation (RAG), and context engineering?
- ❷ **Forms:** What architectural or representational forms can agent memory take?
- ❸ **Functions:** Why is agent memory needed, and what roles or purposes does it serve?
- ❹ **Dynamics:** How does agent memory operate, adapt, and evolve over time?
- ❺ What are the promising frontiers for advancing agent memory research?

Preliminaries

Memory Lifecycle: Formation, Evolution, and Retrieval The dynamics of the memory system are characterized by three conceptual operators.

Memory Formation At time step t , the agent produces informational artifacts ϕ_t , which may include tool outputs, reasoning traces, partial plans, self-evaluations, or environmental feedback. A formation operator

$$\mathcal{M}_{t+1}^{\text{form}} = F(\mathcal{M}_t, \phi_t)$$

selectively transforms these artifacts into memory candidates, extracting information with potential future utility rather than storing the entire interaction history verbatim.

Memory Evolution Formed memory candidates are integrated into the existing memory base through an evolution operator

$$\mathcal{M}_{t+1} = E(\mathcal{M}_{t+1}^{\text{form}}),$$

which may consolidate redundant entries (Zhao et al., 2024), resolve conflicts (Rasmussen et al., 2025; Li et al., 2025l), discard low-utility information (Wang et al., 2025r), or restructure memory for efficient retrieval. The resulting memory state persists across subsequent decision steps and tasks.

Memory Retrieval When selecting an action, agent i retrieves a context-dependent memory signal

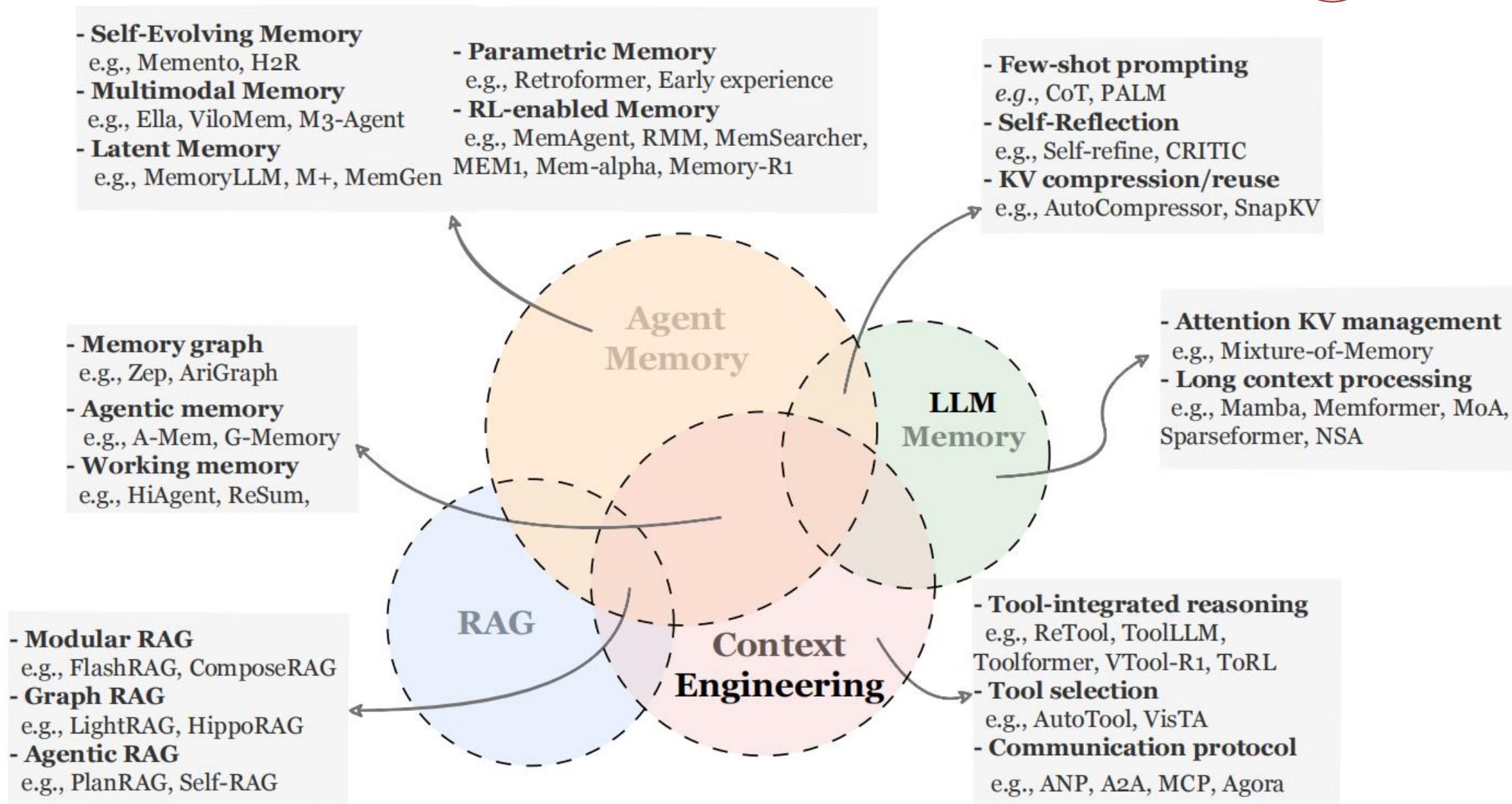
$$m_t^i = R(\mathcal{M}_t, o_t^i, \mathcal{Q}),$$

where R denotes a retrieval operator that constructs a task-aware query and returns relevant memory content. The retrieved signal m_t^i is formatted for direct consumption by the LLM policy, for example as a sequence of textual snippets or a structured summary.

Preliminaries



北京大学
PEKING UNIVERSITY



Three Major Memory Forms

1. **Token-level Memory** (Section 3.1): Memory organized as explicit and discrete units that can be individually accessed, modified, and reconstructed. These units remain externally visible and can be stored in a structured form over time.
2. **Parametric Memory** (Section 3.2): Memory stored within the model parameters, where information is encoded through the statistical patterns of the parameter space and accessed implicitly during forward computation.
3. **Latent Memory** (Section 3.3): Memory represented in the model's internal hidden states, continuous representations, or evolving latent structures. It can persist and update during inference or across interaction cycles, capturing context-dependent internal states.

Form: What Carries Memory? Token-level Memory



北京大学
PEKING UNIVERSITY

Experience



e.g., ExpeL, AWM, ReasoningBank

Chunk



e.g., Nemori, Memo, MemOS

Dialogue



e.g., MemGPT, MemoryBank

Summary

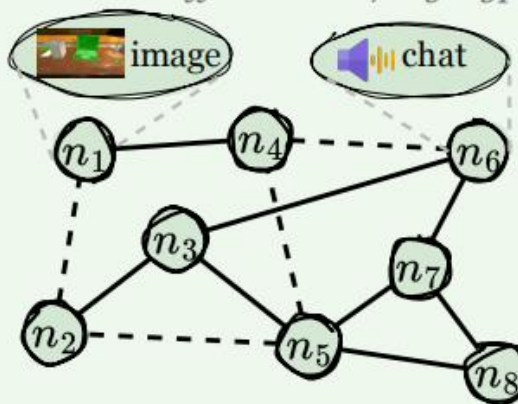


e.g., Think-in-Memory, RMM

(a) Flat Memory (1D)

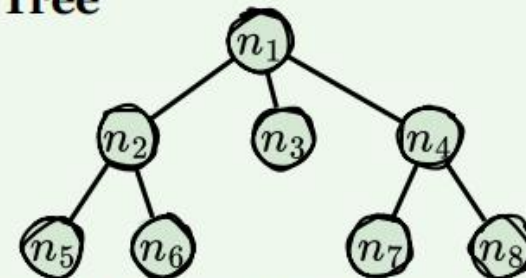
Graph

Memory graphs with
different node/edge types



e.g., A-Mem, Memo^g,
M3-Agent, D-SMART

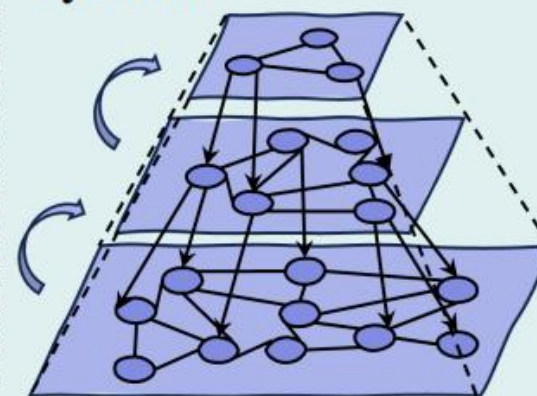
Tree



e.g., MemTree, TME, others

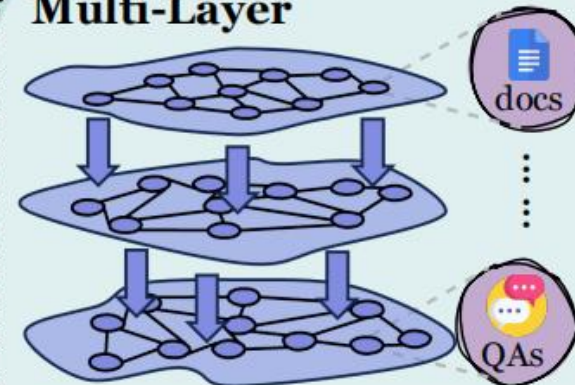
(b) Planar Memory (2D)

Pyramid



e.g., G-Memory, CAM, others

Multi-Layer



HiAgent, HippoRAG, SGMem

(c) Hierarchical (3D)

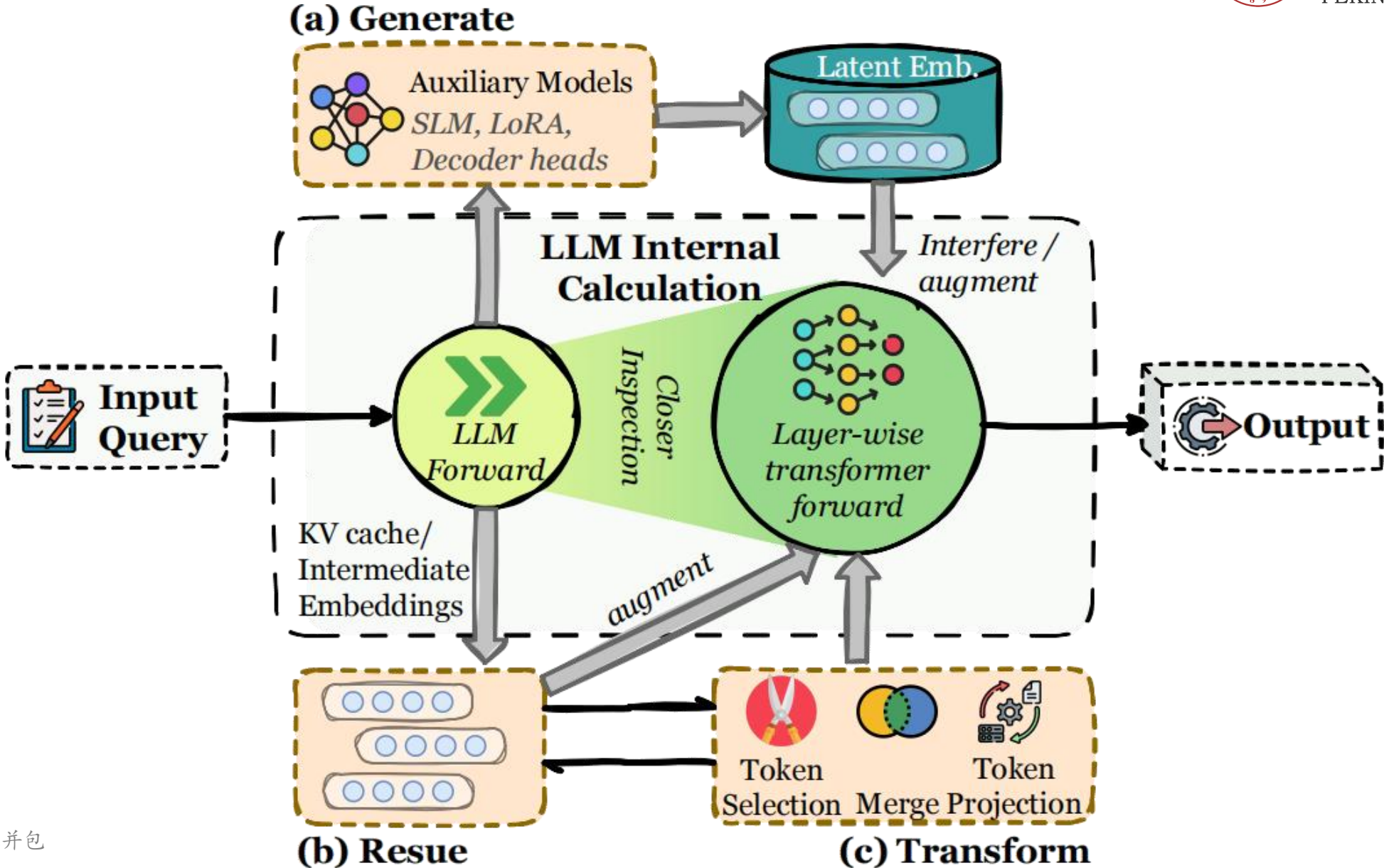
Form:What Carries Memory? Parametric Memory



Two Major Types of Parametric Memory

1. **Internal Parametric Memory:** Memory encoded within the original parameters of the model (e.g., weights, biases). These methods directly adjust the base model to incorporate new knowledge or behavior.
2. **External Parametric Memory:** Memory stored in additional or auxiliary parameter sets, such as adapters, LoRA modules, or lightweight proxy models. These methods introduce new parameters to carry memory without modifying the original model weights.

Form:What Carries Memory? Latent Memory



Form: What Carries Memory?



北京大学
PEKING UNIVERSITY



Token-level Memory

Features:

- Symbolic, addressable, transparent
- Swift adding/deleting/updating

Suitable Applications:



Multi-turn chatbot



Personalized agents



Recommender system



High-stake domains
(law, finance, medical)



Parametric Memory

Features:

- Implicit, abstract, and generalizable
- Slower memory update
- (Typically) better performance gain
- More severe catastrophic forgetting

Suitable Applications:



Role-playing



Reasoning-intensive task



Tasks that require
fundamentally new
capabilities



Latent Memory

Features:

- Implicit, human-unreadable
- Trade-off between efficiency/flexibility
- Convenient modality fusion
- Machine-native and token-efficient

Suitable Applications:



Multimodal memory



On-device or edge deployment



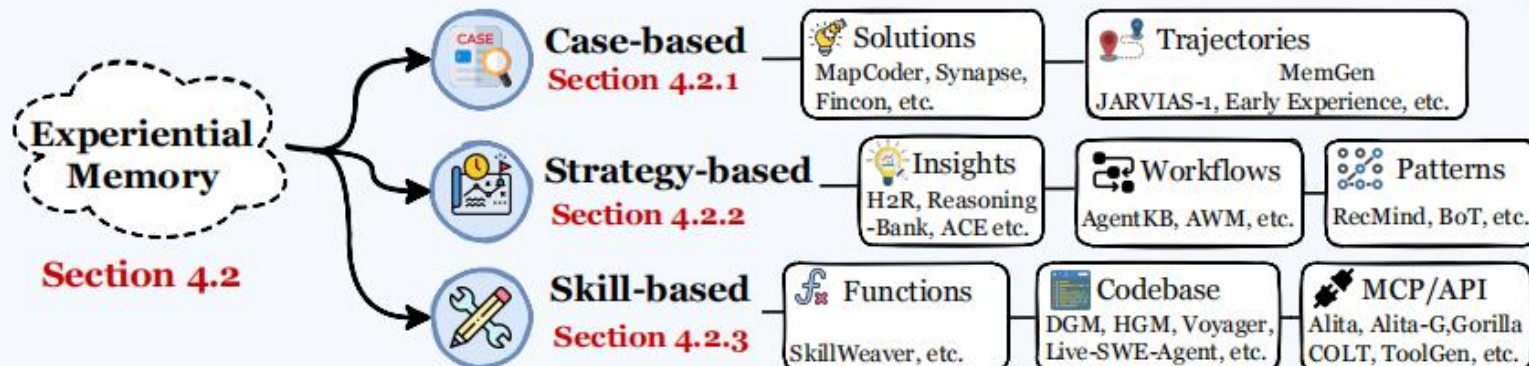
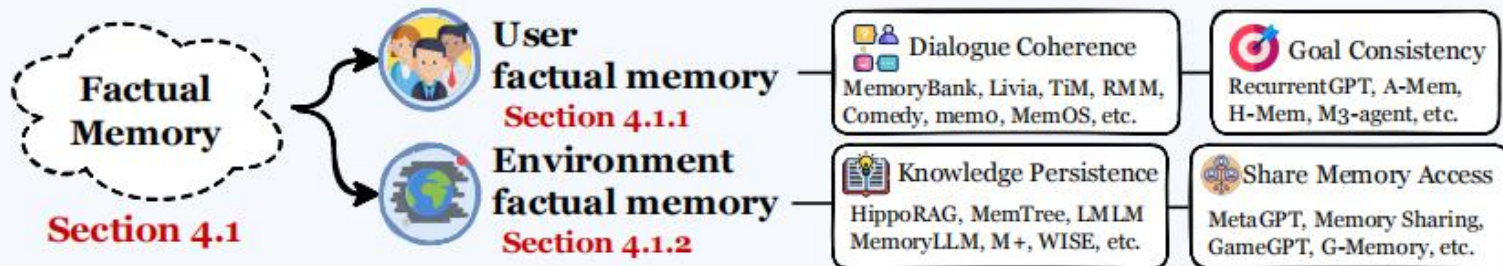
Low-resource or small-data
setting

Functions: Why Agents Need Memory?

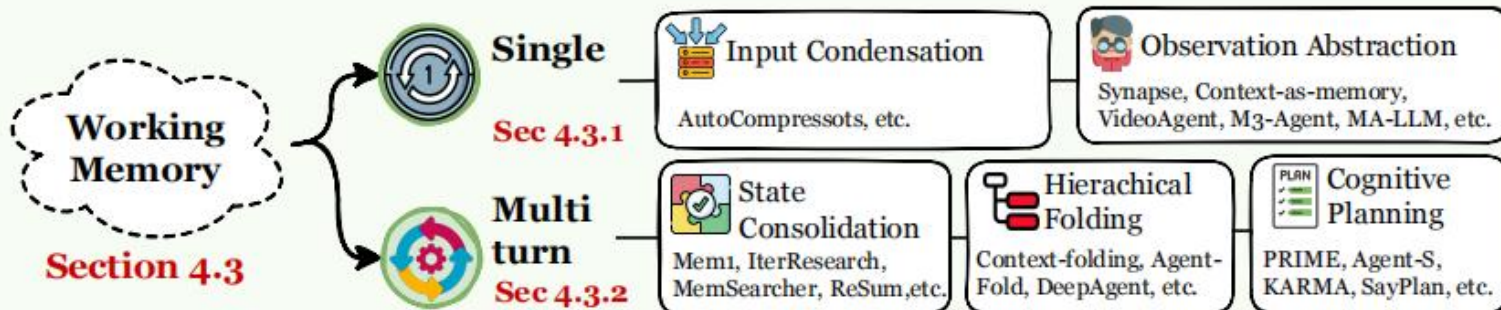


北京大学
PEKING UNIVERSITY

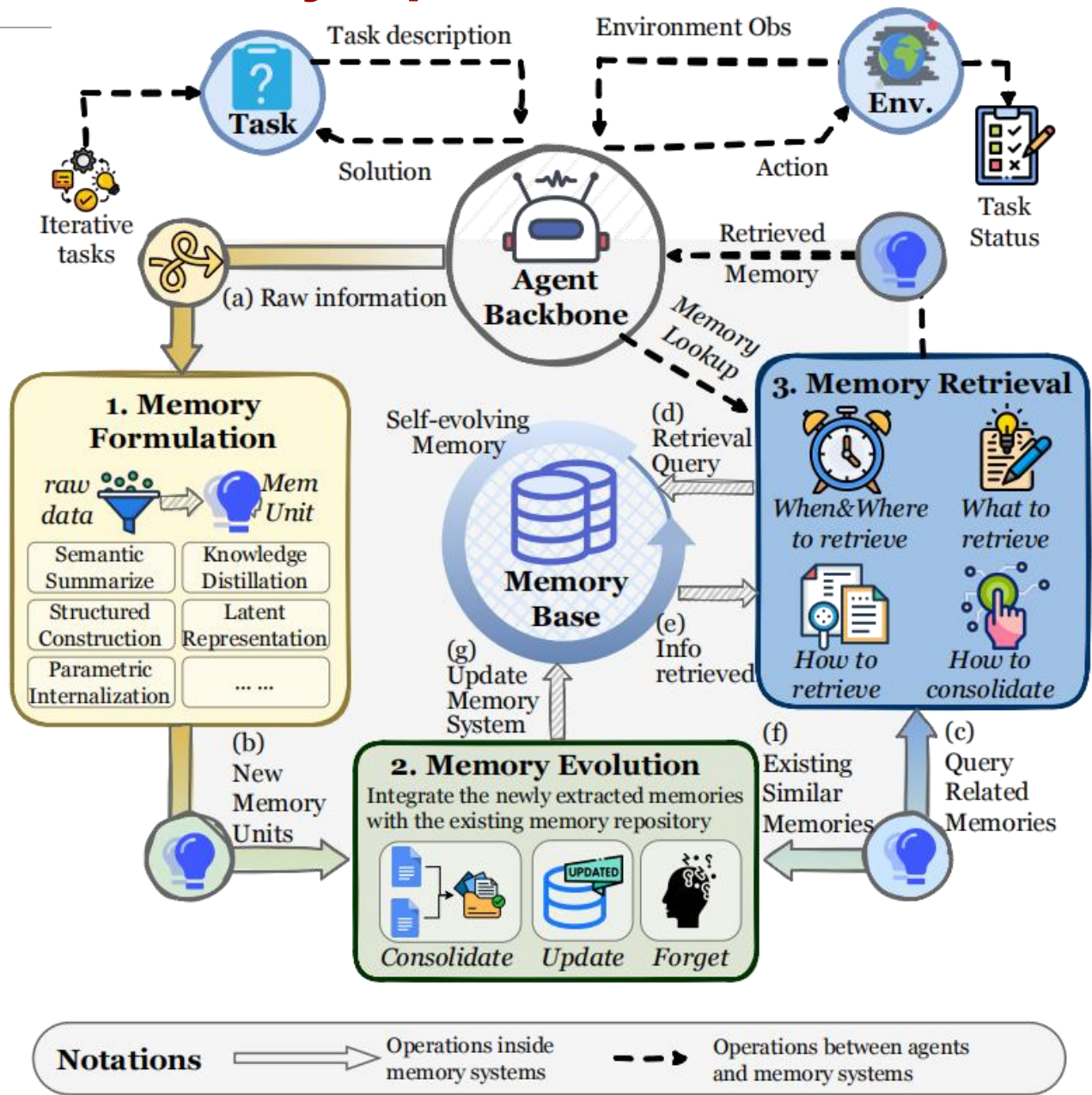
(a) Long-term Memory



(b) Short-term Memory



Dynamics: How Memory Operates and Evolves?



记忆的增删改查

Position and Frontiers

1.生成式记忆：情景自适应、多模态、可学习

2.自动记忆管理：摆脱固定workflow

3.RL融合的记忆设计：RL可以用于记忆动态的哪些方向，如何逐步去除非必要的先验

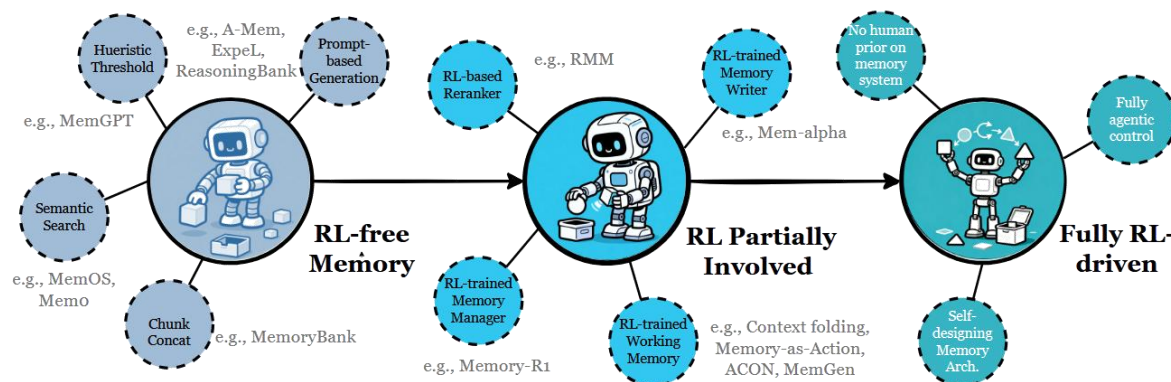
4.多模态记忆：从多模态到全模态

5.多智能体共享记忆：如何跨异构通信

6.世界模型：记忆如何辅助进行世界建模

7.可信记忆：记忆本身的安全性、可验证性

8.类人记忆：哪些角度与人类记忆还存在gap





MemGen: Weaving Generative Latent Memory for Self-Evolving Agents

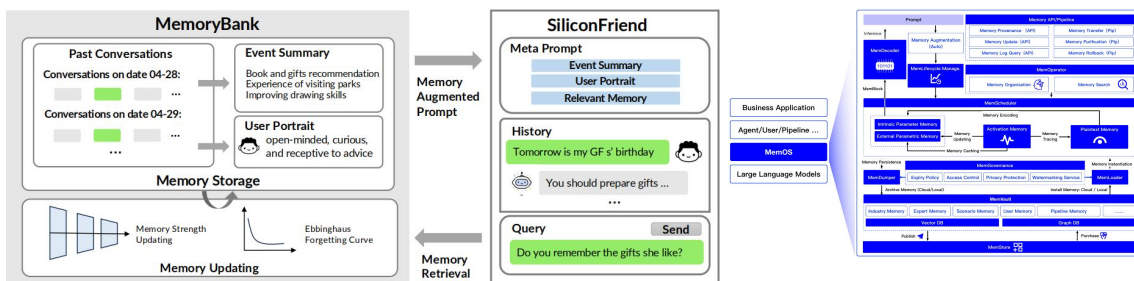
GuibinZhang, MuxinFu, ShuichengYan
National University of Singapore

 **Date:** October 14, 2025

 **Github:** <https://github.com/KANABOON1/MemGen>

Chatbot/Personalized Memory

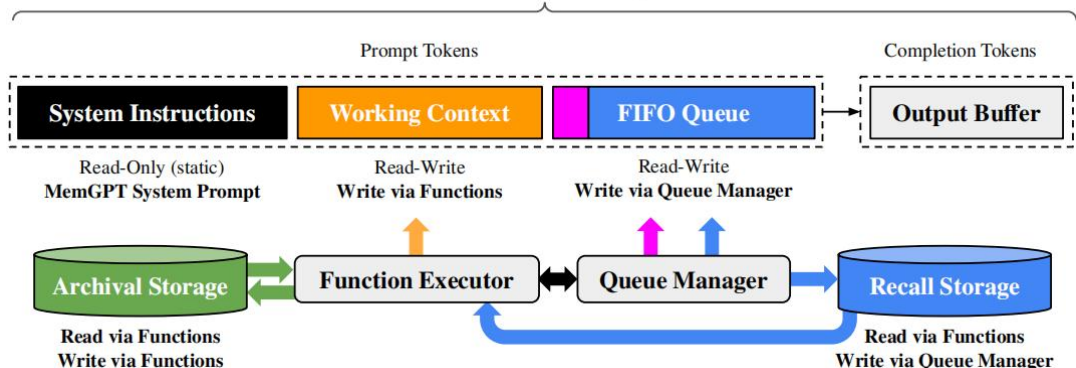
主要关注agent作为chatbot和用户多轮、多会话交互的过程中，如何动态捕获更新用户的信息和偏好



MemoryBank(AAAI 2024)

MemOS(2025)

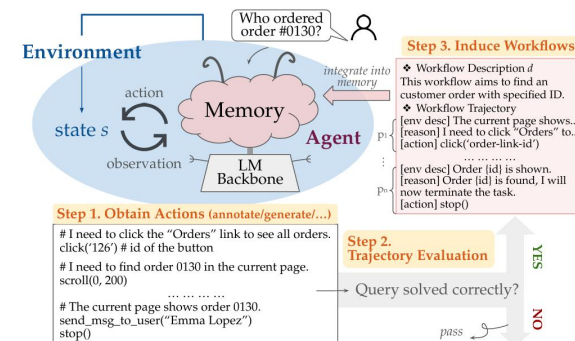
LLM Finite Context Window (e.g. 8k tokens)



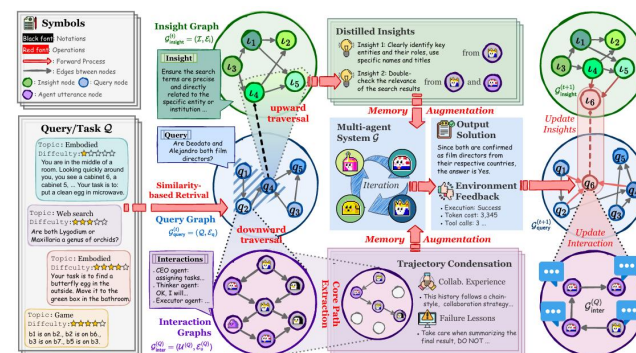
思想自由 兼容并包 MemGPT(2023-10)

Self-Improving Memory

让agent学会怎么利用之前的任务执行轨迹或者经验提升对新任务 (in-domain or out-of-domain)的解决能力

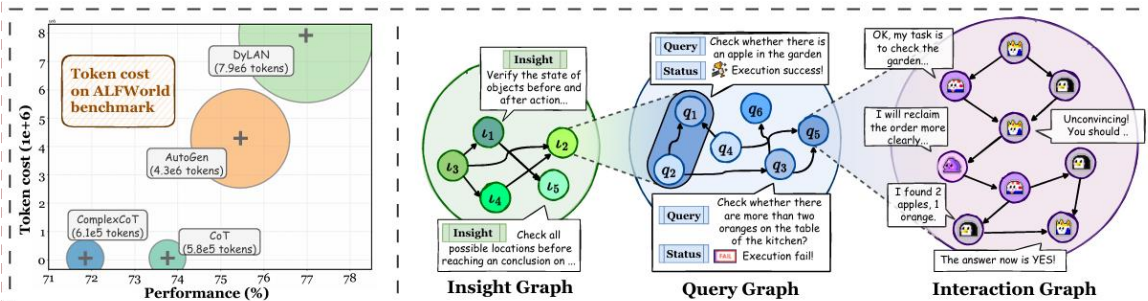


AWM2025



G-Memory(2025)

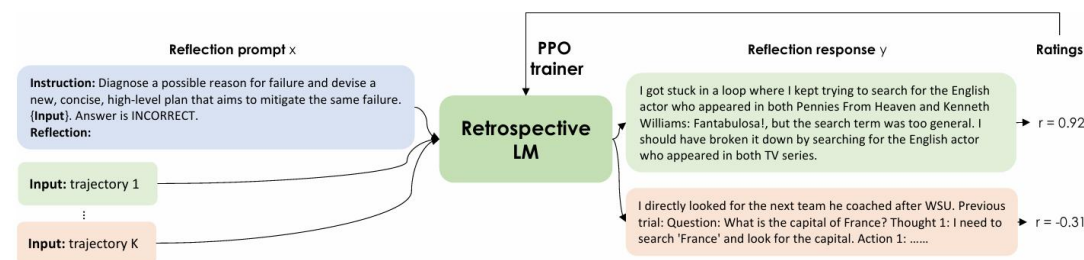
Retrieval-based Memory



往往以外部文字或其他结构化形式存在，比如向量数据库，图数据库等等，经典的工作有MemoryBank, G-Memory、ReasoningBank等

记忆本身以context engineering的形式展现性能存在上限

Parametric Memory



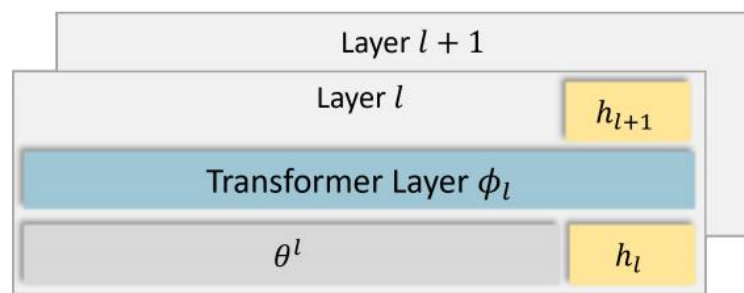
可微调的回顾模型Retrospective Model

将经验直接内化到agent参数内部这个定义很广泛，甚至SFT/GRPO也能算进来；一个经典的例子是retroformer(ICLR 2024)

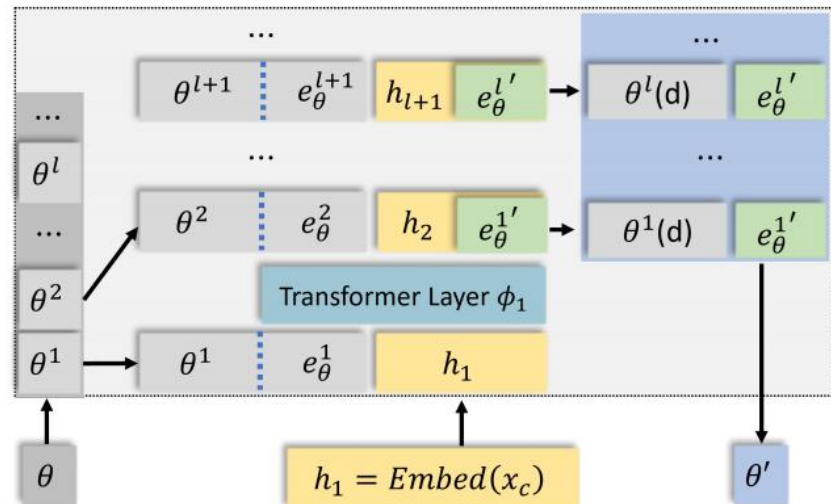
会对LLMagent的参数本身进行训练可能造成灾难性遗忘
PS:RL GRPO/DAPO☑

Latent Memory

每一层中额外添加的一组向量（记忆令牌）

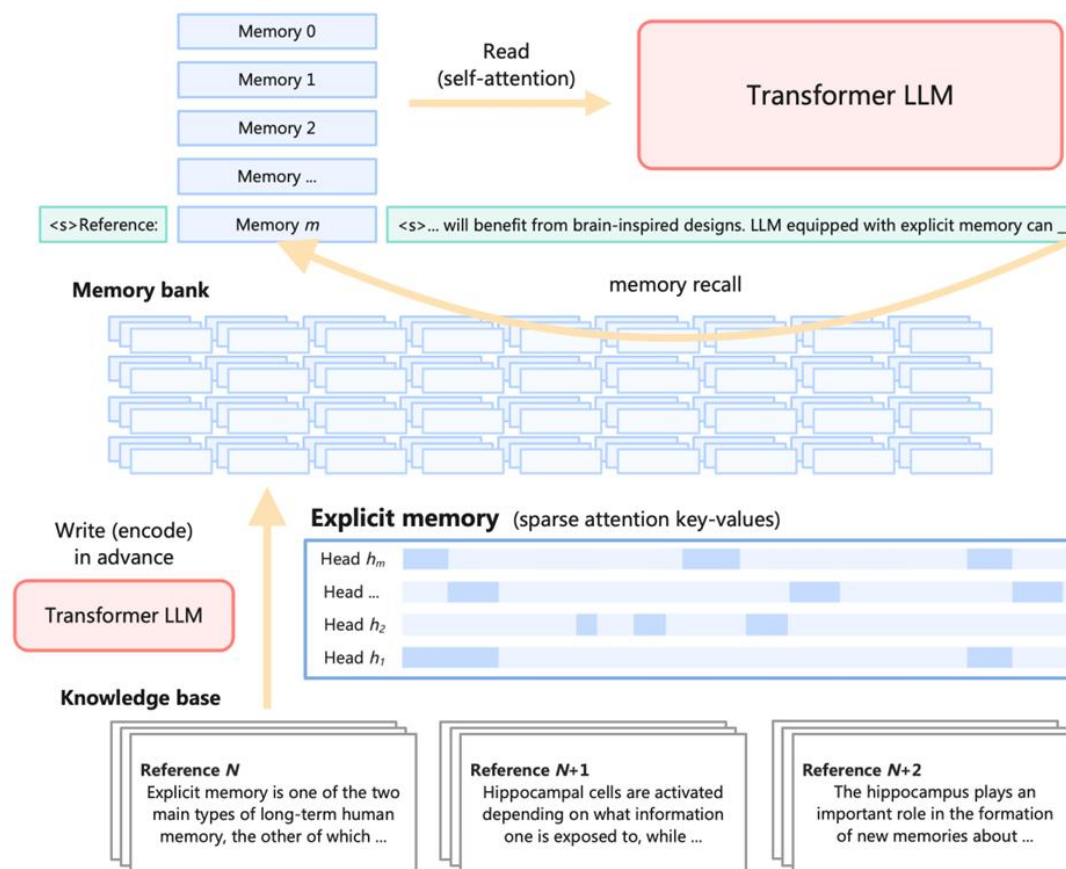


(a) Generation



MemoryLLM(2024)

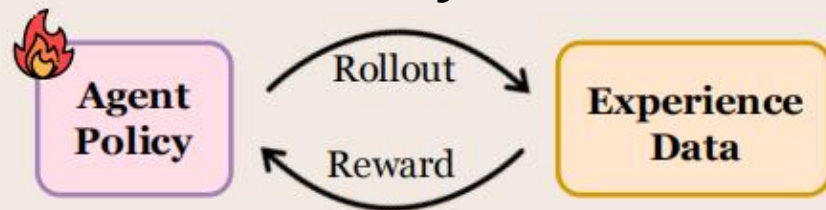
以latent embedding/hidden states作为更加 machine-native的memory载体



Memory³ (JML 2024)

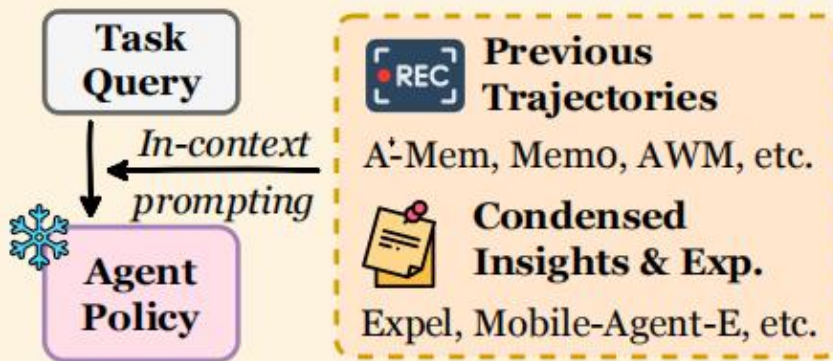
MemGen: 生成式 隐式记忆

Parametric Memory



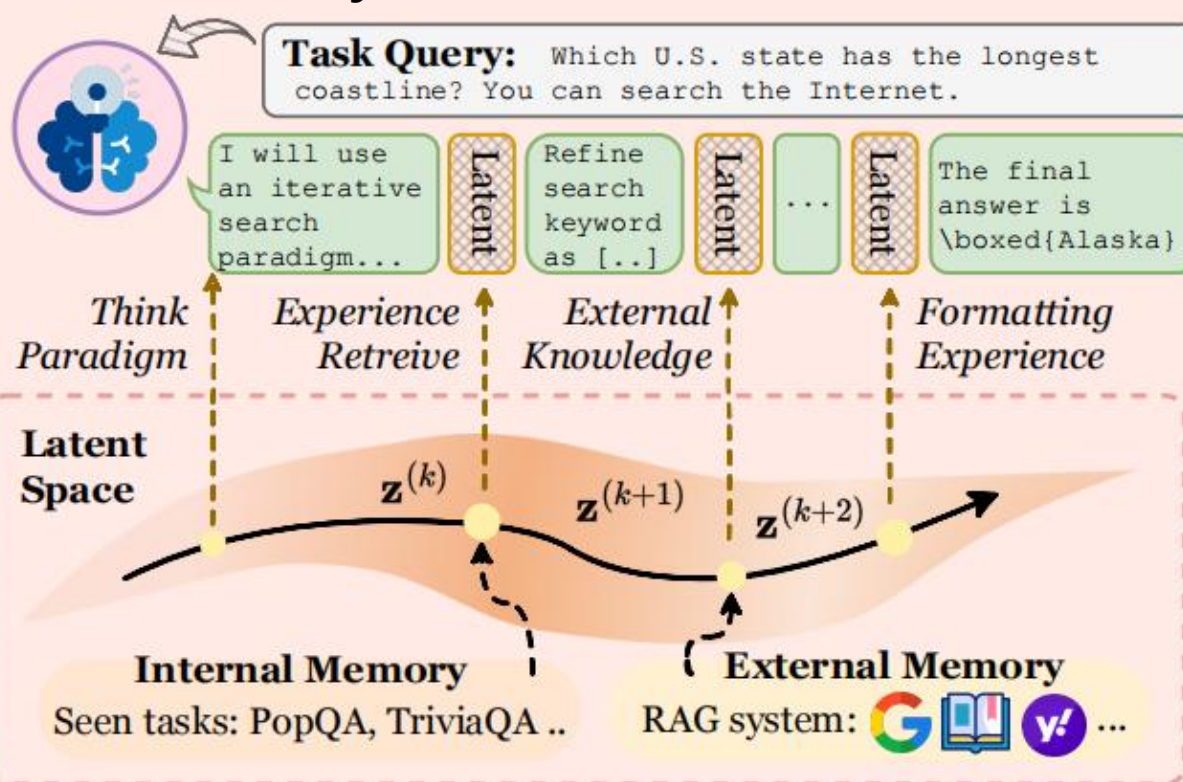
SFT, GRPO, DPO, etc.

Retrieval-based Memory



Agent Memory: 第三种可能

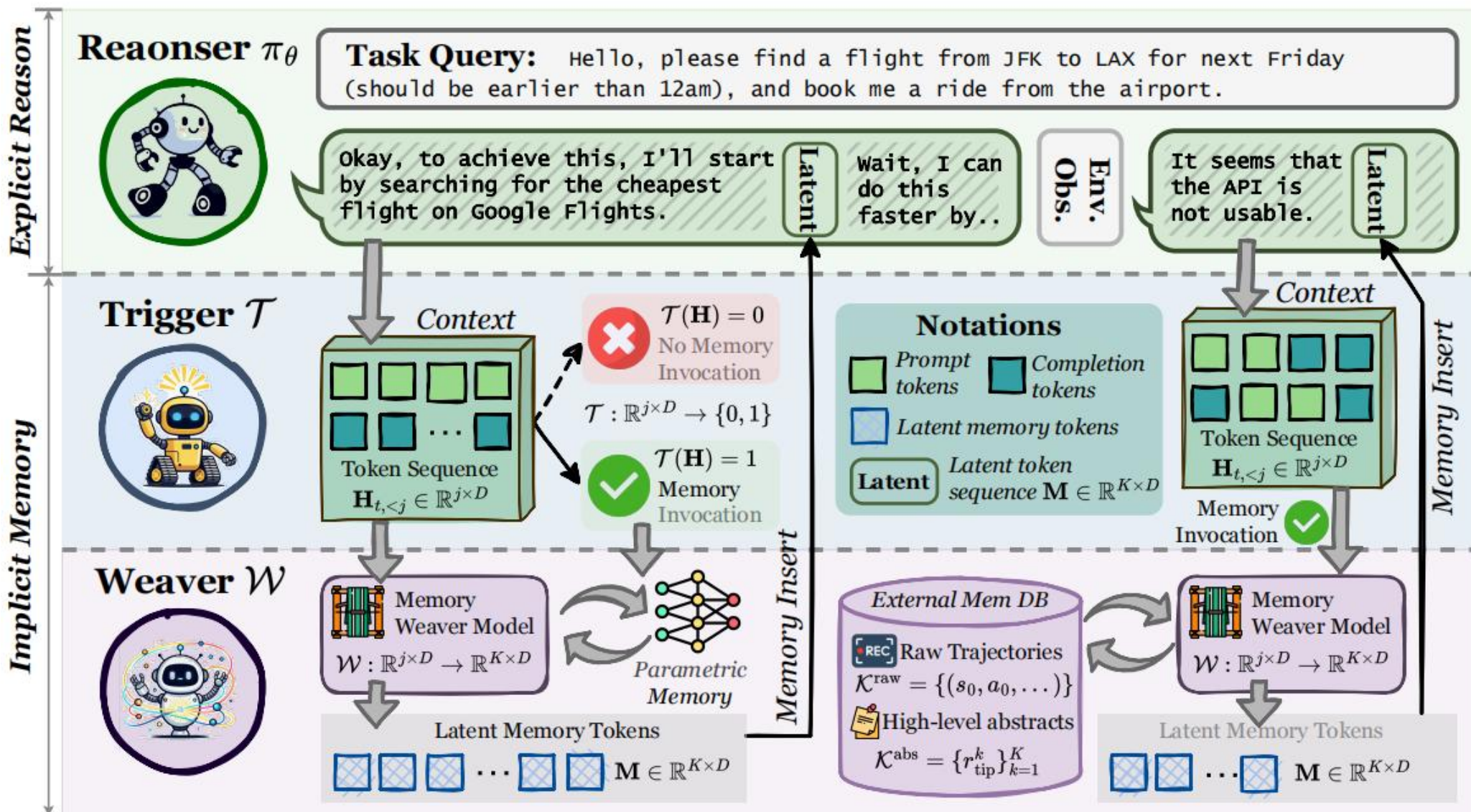
Latent Memory (for MemGen)



方法概述



北京大学
PEKING UNIVERSITY



Memory Trigger [inference]

LLM在解码到第t个token时，memory trigger需要用以判断此处是否显式地启动memory invocation

Memory trigger以当前LLM解码出的所有hidden states作为输入：

$$\mathbf{H}_{t,<j} \in \mathbb{R}^{(j-1) \times d_{\text{model}}}$$

输出判断当前是否进行Memory调用：

$$d_j = \text{Bernoulli}(p_j), \quad p_j = \begin{cases} 0 & \text{if } z_j \notin \mathcal{D}, \\ \mathcal{T}_{\text{trigger}}(\mathbf{H}_{t,<j}) & \text{if } z_j \in \mathcal{D}, \end{cases}$$

D代表语义间隔：逗号，句号，换行等

Memory Trigger [training]

Training Recipe. The memory trigger is trained via reinforcement learning, motivated by the need to balance two competing desiderata: ensuring that critical latent memories are invoked to improve task performance, while avoiding unnecessary or spurious invocations that could disrupt reasoning or incur computational overhead. Given a batch of seen tasks $\mathcal{H} = \{(x_i, \tau_i)\}_{i=1}^N$, the frozen reasoner π_θ generates candidate trajectories while the memory weaver $\mathcal{W}_{\text{weaver}}$ remains fixed. At each activated step, the trigger selects an action $\tilde{d}_j \in \{\text{INVOKE}, \text{SKIP}\}$ and receives a reward $r(\tau_i)$ reflecting the quality of the resulting trajectory with respect to the task objective. To encourage sparse yet strategically critical memory invocation, we introduce a *reward-adaptive penalty*, which discourages unnecessary activations while preserving essential ones, into the objective:

$$\max_{\phi} \mathbb{E}_{\tau_i \sim \pi_\theta, \tilde{\mathbf{d}} \sim \mathcal{T}_{\text{trigger}}^\phi} \left[R(\tau_i) - \lambda \sum_{i,j} \max(0, \tilde{d}_{i,j} - \bar{p}) \right], \quad (8)$$

where $\bar{p}$ is computed as the mean activation probability across high-reward trajectories, *i.e.*, those with reward exceeding the batch median:

$$\bar{p} = \frac{1}{|\mathcal{H}_{\text{high}}|} \sum_{i \in \mathcal{H}_{\text{high}}} \frac{1}{|\tau_i|} \sum_j \tilde{d}_{i,j}, \quad \mathcal{H}_{\text{high}} = \{i : R(\tau_i) \geq \text{median}_k(R(\tau_k))\}, \quad (9)$$

where ensures that $\mathcal{T}_{\text{trigger}}$ learns to invoke memory selectively at key decision points, maximizing task reward while maintaining computational efficiency.

Memory Weaver [inference]

当memory trigger判断需要进行memory调用的时候, memory weaver 被唤醒并根据当前的context动态地编织记忆

Memory weaver 同样以当前LLM解码出所有hidden states作为输入:

$$\mathbf{H}_{t,<j} \in \mathbb{R}^{(j-1) \times d^{\text{model}}}$$

并生成一段长度为K的latent token 序列:

$$\mathbf{M}_t = \mathcal{W}_{\text{weaver}}^{\theta'}(\mathbf{H}_{t,<j})$$

Memory Weaver [training]

Training Recipe. The training of $\mathcal{W}_{\text{weaver}}$ proceeds over a batch of past trajectories $\mathcal{H} = \{(x_i, \tau_i)\}_{i=1}^N$. Distinct from conventional agent tuning, which directly integrates experiential data into the parameters of π_θ (Chen et al., 2025; Yin et al., 2024), **MemGen** internalizes experiential knowledge solely into $\mathcal{W}_{\text{weaver}}$, which ensures that π_θ 's general capabilities remain intact.

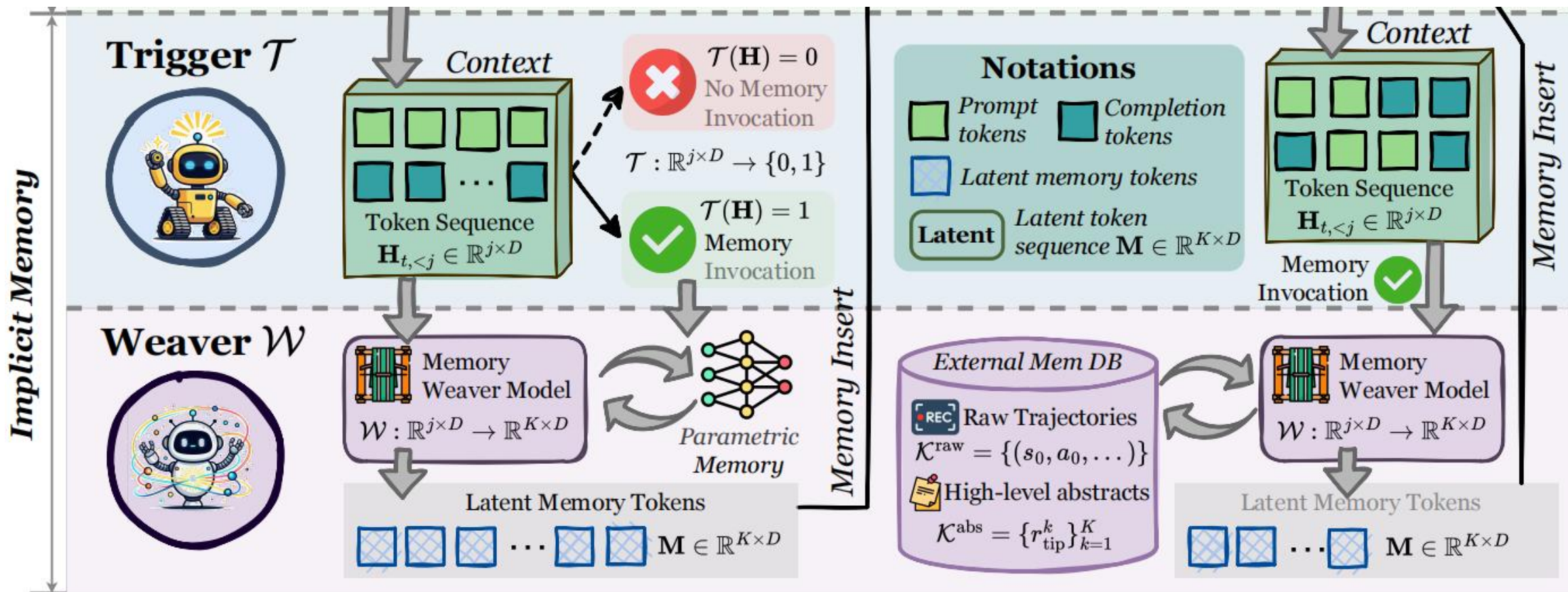
Crucially, this separation makes **MemGen** agnostic to optimization strategies and compatible with diverse LLM backbones. Whether employing supervised **fine-tuning (SFT)** or RL-based objectives such as **GRPO** or **DAPO**, the weaver can be updated under a unified goal: optimizing the generation process of latent memory so as to maximize downstream reward. Formally, let $\Pi_{\theta}^{\mathcal{W}_{\theta'}, \mathcal{T}}(\cdot | x)$ denote the process of rolling out a trajectory for a task x by π_θ in conjunction with weaver $\mathcal{W}_{\theta'}$ and trigger $\mathcal{T}$. Given a reward functional R , the objective updates only θ' by maximizing the expected reward:

$$\max_{\theta_{\text{loa}}} \mathbb{E}_{(x_i, \tau_i) \sim \mathcal{H}} \mathbb{E}_{\tau \sim \Pi_{\theta}^{\mathcal{W}_{\theta'}, \mathcal{T}}(\cdot | x_i)} [R(x_i, \tau)], \quad (10)$$

where the gradients from R are propagated solely to θ' , thereby equipping the weaver to supply precisely the memories that improve end-to-end performance without altering π_θ . **Equation (10)** enables $\mathcal{W}_{\text{weaver}}$ to absorb diverse experiential signals and externalize them as dynamic, context-sensitive latent memories, independent of the architectural or training paradigm of the base reasoner.

Instantiation

Memory weaver和trigger的实现都是LoRA adapter
但这并不代表weaver和trigger只能是LoRA adapter



可以是：一个更小的SLM（譬如reasoner是7b，weaver是1.5b）
其他PEFT方法
也可以是直接复刻reasoner的全量参数进行微调

In this section, we conduct extensive experiments to answer the following research questions:

- (RQ1) Can **MemGen** surpass both parametric and retrieval-based memory?
- (RQ2) Is the memory learnt by **MemGen** generalizable across task domains? And why?
- (RQ3) Can **MemGen** facilitate continual learning and mitigate catastrophic forgetting?
- (RQ4) Does **MemGen** implicitly evolve human-like memory hierarchy?

Dataset:

- TriviaQA, PopOA, GPQA, ALFWorld, AQuA, MATH, KodCode, BigCodeBench...

Baselines

- Prompt-based methods : Vanilla model and CoT
- Parametric memory : SFT/GRPO/REINFORCE/REINFORCE++/Agent-FLAN
- Retrieval-based memory : MemoryBank, ExpeL, AWM
- Latent Reasoning: SoftCoT, Co-processor

RQ1: Performance



Table 1 Results on **SmolLM3-3B** and **Qwen3-8B**. All values represent the performance metric for each task (e.g., accuracy %). We highlight the **best** and **second best** results.

Backbone	Method	ALFWorld	TrivialQA	PopQA	KodCode	BigCodeBench	GPQA	GSM8K	MATH
SmolLM3-3B	Vanilla	18.96	10.47	8.23	37.05	35.96	9.35	47.63	16.22
	CoT	17.60	12.88	9.95	38.45	39.42	20.70	58.91	56.33
	SFT	32.36	55.25	37.22	59.25	40.79	19.70	63.48	45.65
	GRPO	55.35	65.88	45.16	68.48	72.44	22.73	80.03	61.23
	REINFORCE	53.13	63.20	46.81	65.53	67.14	23.44	82.03	58.75
	REINFORCE++	53.95	63.20	44.10	65.90	68.80	22.73	81.50	59.89
	Agent-FLAN	34.00	56.70	39.50	56.80	37.20	17.80	59.60	36.84
	ExpeL	36.18	46.20	28.16	51.14	40.22	15.15	56.23	38.11
	MemoryBank	32.80	43.30	25.81	44.50	31.80	10.20	58.30	43.53
	AWM	40.50	49.80	29.60	-	-	-	-	-
	SoftCoT	35.03	50.38	34.90	59.20	39.10	17.22	56.34	44.62
	Co-processor	38.36	53.28	38.96	56.25	45.40	20.10	57.60	38.81
	MemGen _{SFT}	50.60	68.13	42.34	62.65	42.99	26.75	70.42	57.44
	MemGen _{GRPO}	63.60	79.30	58.60	72.85	74.24	25.20	83.47	63.65
Qwen3-8B	Vanilla	58.93	52.18	34.13	49.10	33.33	38.18	89.48	79.82
	CoT	57.10	53.80	33.20	51.25	35.59	35.15	87.67	78.24
	SFT	83.59	74.55	51.12	64.75	41.33	40.33	90.76	81.35
	GRPO	85.60	76.15	58.90	73.35	70.24	39.54	92.30	83.54
	REINFORCE	82.10	75.22	57.96	72.11	70.20	37.12	91.25	83.27
	REINFORCE++	84.80	75.90	58.30	72.90	71.88	37.68	91.90	85.24
	Agent-FLAN	80.32	70.32	50.08	62.99	43.40	39.50	87.60	80.05
	ExpeL	78.97	65.54	40.33	57.20	34.23	35.15	86.20	77.40
	MemoryBank	70.41	60.56	41.60	56.39	40.61	35.66	90.35	80.35
	AWM	80.33	69.30	43.69	-	-	-	-	-
	SoftCoT	75.60	59.42	39.42	63.28	38.27	39.60	86.30	76.23
	Co-processor	73.28	61.42	45.55	64.90	42.19	39.15	76.23	79.20
	MemGen _{SFT}	85.82	77.22	54.65	66.15	40.35	43.23	91.25	83.30
	MemGen _{GRPO}	90.60	80.65	62.30	76.16	75.56	40.24	93.20	88.24

RQ2: Generalization

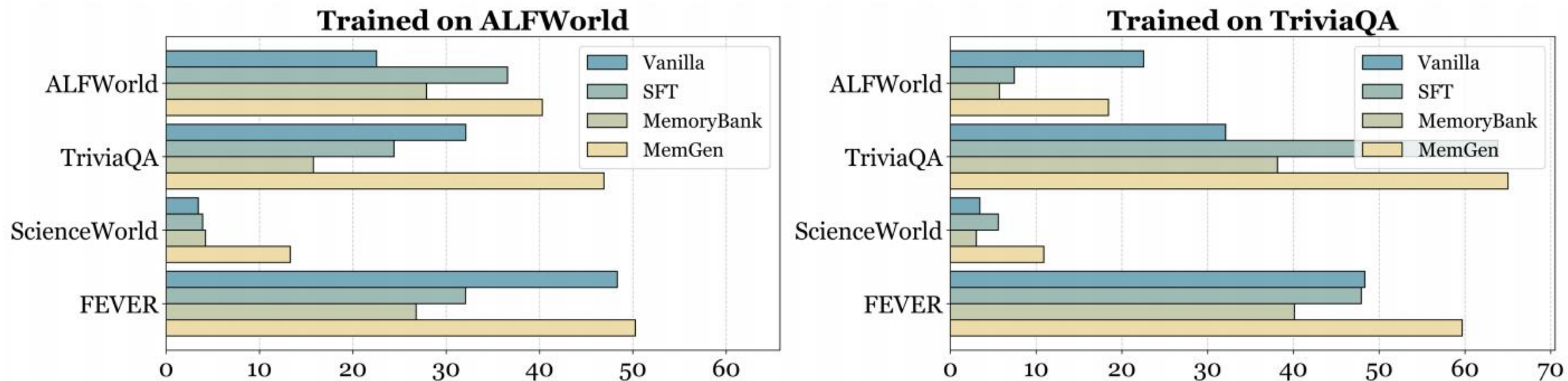


Figure 3 The generalization study of **MemGen**. We train **MemGen**_{SFT} on one dataset (ALFWorld or TriviaQA) and evaluate it on four datasets (TriviaQA, ALFWorld, ScienceWorld, and FEVER).

RQ2: Why Generalization?

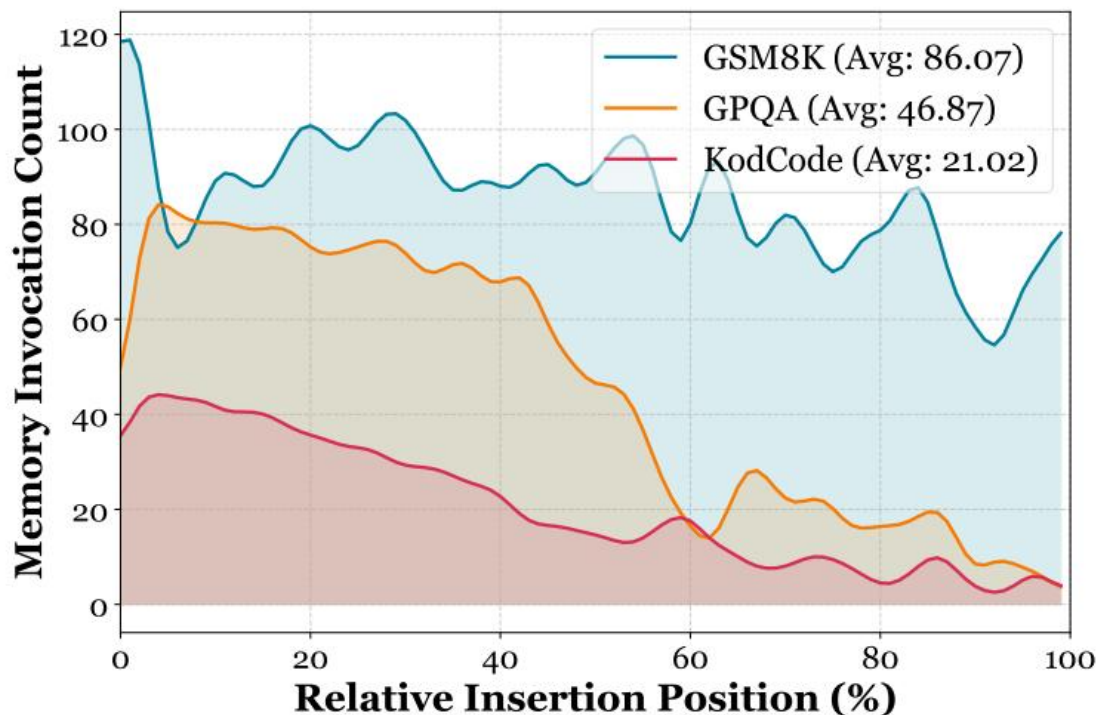
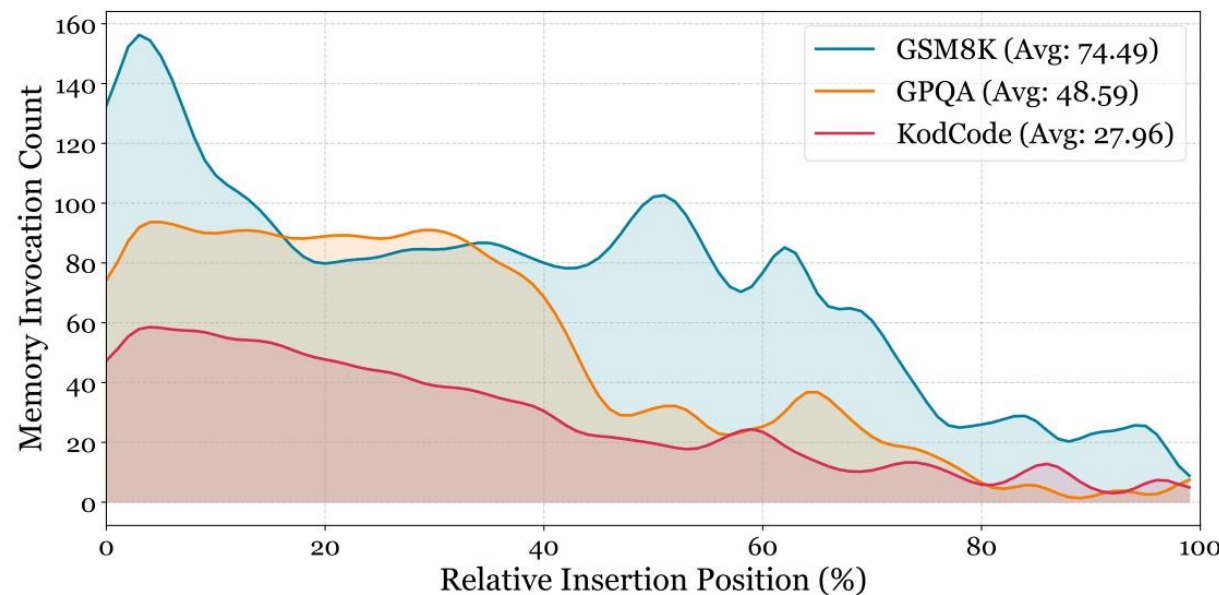


Figure 4 Memory invocation frequency across benchmarks at inference (trained on **MemGen** $\text{sFT} + \text{Qwen3-8B} + \text{GSM8K}$).



SmoLLM3-3B

Remark:

MemGen的trigger可以天然地根据context学习到, 什么时候召唤weaver产生记忆对当前是有益的而什么时候可能是没有帮助的

RQ3: Continual Learning

Table 4 Continual learning results of Qwen2.5-1.5B-Instruct across four reasoning and programming datasets (AQuA, GPQA, GSM8K, KodCode). The model is sequentially trained on each dataset (AQuA $\rightarrow$ GPQA $\rightarrow$ GSM8K $\rightarrow$ KodCode), and after each training stage, evaluation is conducted on all four benchmarks.

Trained On	Method	AQuA	GPQA	GSM8K	KodCode
AQuA	Vanilla	41.34	11.62	39.51	24.55
	SFT	42.52	16.67	42.10	18.20
	ExpeL	41.73	12.67	40.16	16.30
	MemGen _{SFT}	43.31	19.70	39.80	19.55
GPQA	SFT	38.55	17.17	45.74	18.50
	ExpeL	37.24	14.35	42.67	15.20
	MemGen _{SFT}	39.85	20.72	47.96	28.80
GSM8K	SFT	33.46	13.13	52.31	19.45
	ExpeL	34.89	12.42	48.78	13.65
	MemGen _{SFT}	38.43	21.72	55.67	19.75
KodCode	SFT	28.61	2.53	24.14	54.10
	ExpeL	27.14	6.23	31.44	48.35
	MemGen _{SFT}	40.34	20.09	53.72	52.95

RQ4: Interpretation

Case 2

MODEL INPUT:

Write an efficient and correct Python function to solve the following problem. Question: Write a python function to find the first maximum and the second maximum number in an array. If the array has less than two unique elements, return 'None' for the missing values.

MODEL OUTPUT:

```
[keyword-kindergarten-slide]def find_first_max_second_max(arr):
```

```
    """
```

```
[.keywordetical] Returns the first and second maximum numbers in the array.
```

```
    If the array has less than two unique elements,[.keyword_pick] returns None for the missing values.
```

```
[.keyword_pick] """
```

```
    if len(arr) < 2:
```

```
[.keyword_pick] return None,[.keyword_pick] None
```

```
[.keywordetical] unique_elements = set(arr)
```

```
[.keyword_pick] if len(unique_elements) < 2:
```

```
[.keywordetical] return None,[.keyword_pick] None
```

```
[.keywordetical] first_max = max(unique_elements)
```

```
    second_max = max(unique_elements, default=None)
```

```
    return first_max, second_max<|im_end|>
```

代码

RQ4: Interpretation

Case 2

MODEL INPUT:

Solve the math problem with proper reasoning, and make sure to put the FINAL ANSWER inside `\boxed{}`.

Question: Henry made two stops during his 60-mile bike trip. He first stopped after 20 miles. His second stop was 15 miles before the end of the trip. How many miles did he travel between his first and second stops?

MODEL OUTPUT:

[..... a even even even... even] Henry traveled $60-15=<<60-15=45>>45$ miles before his second stop.

[-android.] He traveled 20 miles before his first stop, [ecies.] so he traveled $45-20=<<45-20=25>>25$ miles between his first and second stops. `\boxed{25}`<|im_end|>

[even.] The final answer is 25<|im_end|>

Case 3

MODEL INPUT:

Solve the math problem with proper reasoning, and make sure to put the FINAL ANSWER inside `\boxed{}`. Question: A basket contains 25 oranges among which 1 is bad, 20% are unripe, 2 are sour and the rest are good. How many oranges are good?

MODEL OUTPUT:

[. even.. pre. even even even... even] The number of unripe oranges is $20/100*25 = <<20/100*25=5>>5$

[-animate.] The number of sour oranges is 2

[-login.] The number of bad oranges is 1

[-login.] The number of good oranges is $25 - 5 - 2 - 1 = <<25-5-2-1=17>>17$ `\boxed{17}`<|im_end|>

数学

RQ4: Interpretation

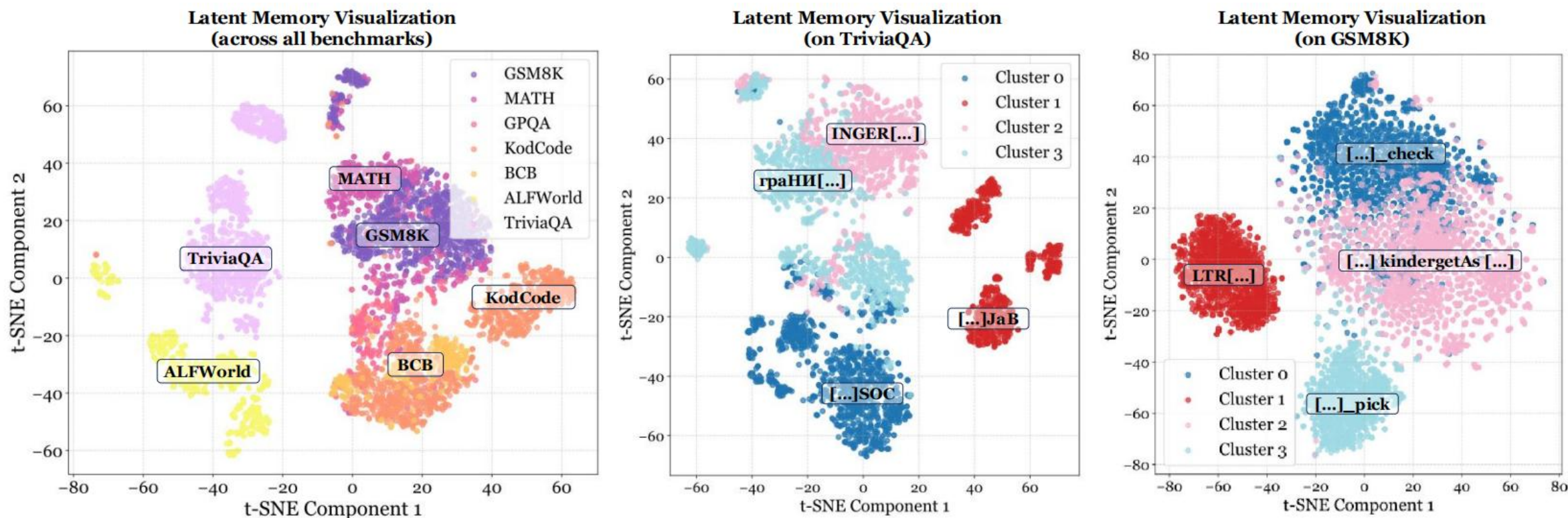
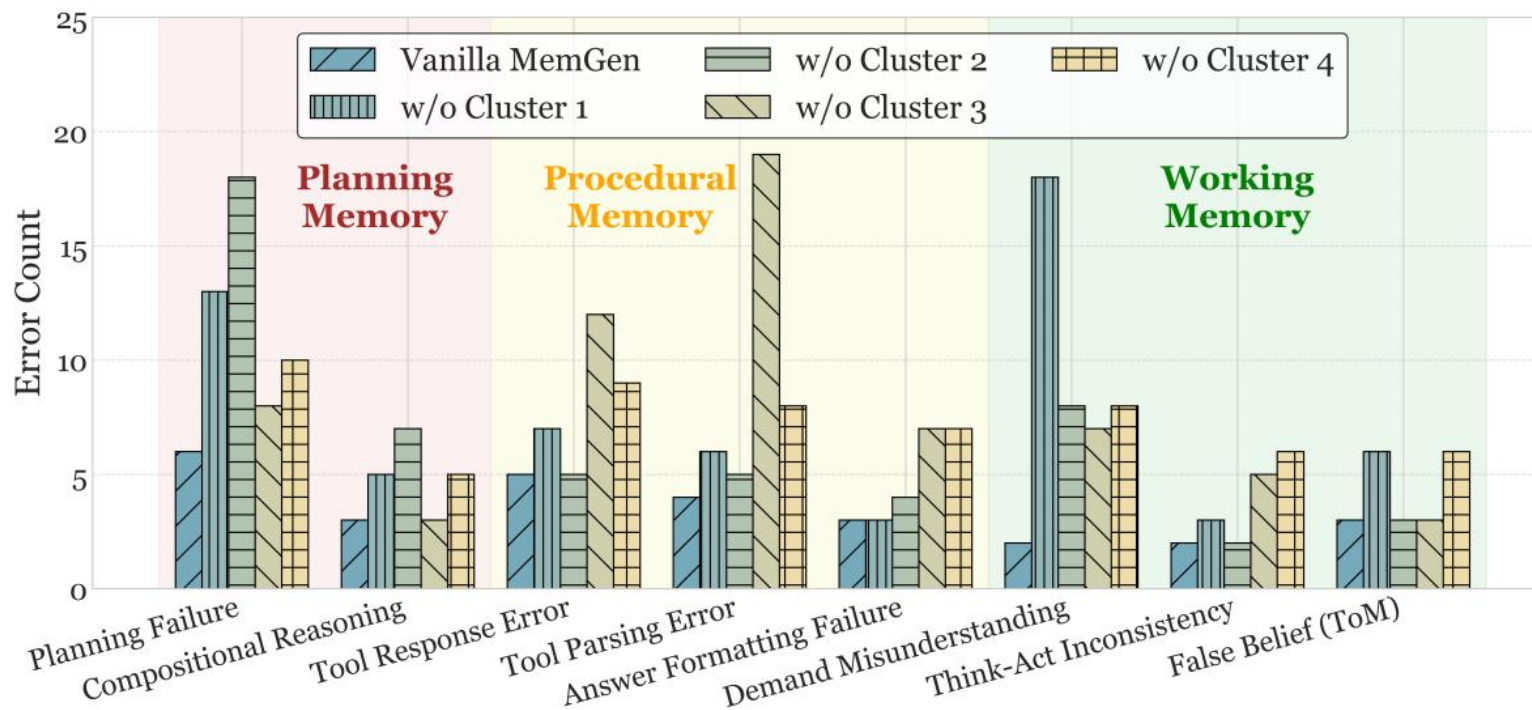


Figure 5 (*Left*) t-SNE visualization of latent memories generated by **MemGen** + Qwen3-8B across datasets; (*Middle and Right*) Latent memory visualization within the TriviaQA and GSM8K datasets, clustered using K-means. The text at each cluster center represents the common pattern shared by many memory sequences in the cluster, such as Cluster 0 in GSM8K, where many sequences end with “_check”.

RQ4: Interpretation



- **规划记忆 (Planning Memory):** 负责高层任务规划与策略推理。移除Cluster2后，规划与组合推理失败显著增加，说明该簇对引导LLM代理的决策与推理排序至关重要。
- **程序性记忆(Procedural Memory):** 包含任务相关的操作知识，如工具使用与格式技能。Cluster 3承担此角色，移除后会明显增加工具响应错误、解析失败和答案格式问题。
- **工作记忆 (Working Memory):** 负责保留并有效利用上下文以维持推理一致性。Cluster 1和4承担此功能；例如，移除Cluster 1的latent memory token会导致任务理解偏差与思考行动不一致现象更频繁发生。

Latent reasoning/memory/communication 是一种更加 machine-native 的表征形式

MemGen 提出了一种动态生成式记忆框架，通过解耦设计的 Memory Trigger（记忆触发器）和 Memory Weaver（记忆编织器），在不修改主模型参数的前提下，赋予 LLM Agent 类似人类的记忆与推理交织能力。

Trajectory→Latent Memory→RL Update→Better Latent Memory

训练完以后Memory Weaver参数固定，能不能让weaver自己生长？

如何区分有用和没用的memory？

奖励有用的，惩罚没有用的，让memory成为policy的一部分；增加forgetting policy,奖励下降。

Trajectory→Latent Memory→World State

怎么样从记住经验，变成记住为什么发生（规律）

Trajectory→Latent Memory→Rule Extraction→Latent Rule Memory